

Final Report for the LLM Management System HFT Optimizer Project

Team Members: Alejandro Chavez Ramos (ajc21@illinois.edu), Rushabh Shetty (shetty6@illinois.edu)

1. Summary

This project is a Streamlit application built to optimize high frequency trading systems through integrated data collection, live monitoring, AI driven performance analysis, hardware upgrade advice and configurable AI models. The design follows a tab based interface for clear workflow from information gathering to actionable recommendations.

2. Introduction

The entry point is the file main.py. It imports all tab render functions and the AI config loader. It sets the page to wide layout and creates the six primary tabs labeled Data, Monitor Data, Trends, Performance, Upgrade and AI Settings. Session state is initialized with the loaded AI configuration from ai_config.py which reads or creates ai_config.json with defaults for model temperature and other parameters.

3. Tabs

3.1. Data Tab Description and Usage

In the Data tab implemented by the function render_collect_data_tab in collect_data_tab.py the app first verifies the presence of sections_config.xlsx. Pressing the Collect System Information button loads the Excel sheet using load_sections from data.py. This function reads the Sections sheet applies column cleaning and builds a nested dictionary of sections and subsections with their commands. Each command is executed via subprocess.check_output with interface name substitution from get_default_interface.

Successful outputs and failures are captured in the sections session state dictionary. A summary is built and the full raw report is displayed in an expander with a download button for system_info.txt. Users start here to gather baseline hardware and software details before proceeding to analysis tabs.

For Redfish BMC integration in the same tab users enable BMC queries enter IP port credentials and test the connection with test_redfish_connection which calls fetch_redfish_endpoint to query the root and system endpoints handling ID discovery for Systems and Chassis.

Upon successful test the Collect button triggers collect_redfish_sections. This function iterates selected sections such as BIOS, Processors, Memory PCIeSlots and custom endpoints. For BIOS it uses get_redfish_bios which fetches the Bios endpoint and extracts attributes. Other sections use fetch_redfish_endpoint with appropriate paths. Results are stored in redfish_data session state and displayed with item counts or setting groups computed by get_redfish_groups.

Raw JSON is available in expanders and full data can be downloaded as JSON or cleared. This allows users to incorporate accurate server management data from BMC for more precise AI recommendations.

3.2. Monitor Data Tab Description and Usage

The Monitor Data tab rendered by `render_monitor_tab` in `monitor_tab.py` begins by loading dynamic metrics via `load_dynamic_df` from `data.py`. This filters rows where `Type` is `dynamic_single` substitutes the network interface into commands computes numeric thresholds and units. Users can select all or deselect metrics updating the `monitored_metrics` list in session state.

Checkboxes show current thresholds. An expander holds the AI Assistant for Threshold Recommendations which calls `get_ai_threshold` from `ai.py`. This function builds monitored metrics context with commands current snapshot values and existing thresholds then renders a chat interface using `render_ai_chat` for threshold advice.

Live monitoring is controlled by start and pause buttons. When running a fragment loop executes. For each monitored metric the prebuilt command is run with subprocess to get the current numeric value. Threshold breaches are detected and tracked in `active_alerts`. Values are appended to in memory `monitor_history` capped at 1000 points and written to `monitoring_history.csv` for persistence.

A table shows current values with status icons. Active breaches are listed with timestamps. Selected metrics for display get live line charts updated each interval. Stopping clears live state but preserves CSV history. A download button provides the full CSV. Users utilize this tab for ongoing surveillance of critical HFT metrics with automatic alerting and historical logging to CSV for later review in the Trends tab.

3.3. Trends Tab Description and Usage

The Trends tab uses `render_history_tab` from `history_tab.py`. It checks for `monitoring_history.csv` and loads it with pandas converting timestamp column. Available metrics are listed in a selectbox. For the chosen metric a time range slider allows zooming if sufficient data points exist. The filtered view is plotted as a line chart. Four metric widgets display min max average and standard deviation for the visible range. This tab enables users to analyze long term patterns in monitored metrics to identify stability issues or performance trends over time.

3.4. Performance Tab Overview

The Performance tab is defined in `performance_tab.py` by `render_performance_tab`. It first checks prerequisites then presents four sub tabs for targeted optimizations. Their main objective is for users to select an HFT profile run the analysis, review AI suggestions, select safe ones and generate a custom tuning script for immediate application. Chat discussion is provided for each sub tab.

3.4.1. OS Config Sub Tab

The OS Config sub tab calls `render_os_config`. Profile selection comes from `get_available_hft_profiles` in `data.py`. Optional Redfish inclusion leads to `perform_hft_analysis` in `ai.py`. Inside `perform_hft_analysis` the function assembles full hardware summary via `build_system_profile` which extracts key sections like CPU, NIC, Memory. It filters to profile specific sections builds detailed text with `build_full_raw_text` and takes a dynamic snapshot with `take_ai_snapshot`. Context is previewed in an expander allowing optional manual text area edit for the prompt sent to AI.

The run button constructs a detailed system prompt enforcing JSON only output with specific fields for analysis and recommendations list. It then invokes `render_structured_ai_task`. The `render_structured_ai_task` function creates a status container runs streaming completion via `litellm` with the system prompt and a force JSON user message. It accumulates the response displays a typing cursor then attempts `json.loads` on the full text or falls back to regex search for

the first JSON object. If successful it extracts analysis markdown and recommendations array. The result is saved to session state under the provided result_key with timestamp and model info. The status updates to complete and triggers rerun.

Back in the sub tab the saved last_analysis is rendered with success message full analysis expander recommendation cards showing title impact risk why hft description and commands. Checkboxes select recommendations for script generation. The generate_tuning_files function builds a bash script with user confirmation prompts for each selected rec and a markdown report. A discussion section calls render_ai_chat with a prompt summarizing the recommendations and hardware profile for follow up questions.

3.4.2. BIOS Sub Tab

The BIOS sub tab follows an analogous pattern in render_bios calling perform_bios_analysis. This function extracts BIOS related sections using get_bios_context which keyword matches titles and outputs for firmware power states C states ASPM. Redfish BIOS attributes are grouped if enabled.

The AI prompt is tailored for BIOS JSON structure including menu paths reboot requirements and impact in microseconds. Display and checklist generation mirror the OS flow with generate_bios_checklist producing a markdown checklist. This helps users identify and document firmware changes that reduce latency with clear reboot and risk indicators.

3.4.3. Compiler Sub Tab

Compiler allows users to scan their HFT project code to receive optimized compiler flags that improve execution speed. The sub tab requires a project path, which is detected using scan button, invoking detect_build_system from data.py. This walks the directory tree identifies build files like Makefile or CMakeLists.txt extracts current CFLAGS via regex parses compiler version and detects hot paths by scanning source files for HFT keywords such as order, trade, latency, capturing surrounding lines and function names. These parameters are customizable under 'Advanced hot-path detection settings'. That is, users can increase the number of lines to include, keywords to look for, character limit and number of hot-paths to include in the analysis.

With build context available perform_compiler_analysis is called. It builds context preview allows manual edit then prompts AI for compiler flags recommendations JSON with rebuild commands. Results display current versus recommended flags rebuild command and generate compiler_tune.sh via generate_compiler_tuning_script which creates a guided bash rebuild script.

3.4.4. Application Code Sub Tab

Application Code allows line by line code review for latency critical sections in trading applications. The sub tab render_application_code uses the same scan for hot paths then calls perform_application_code_analysis. Similarly, users can edit the detection parameters under 'Advanced hot-path detection settings'.

The prompt requests line specific patches in diff format, explain why the suggested patch, its impact and why how it affects high frequency trading. Display includes copy patch buttons and generates apply_patches.sh that writes numbered patch files and applies them interactively.

3.5. Upgrade Tab Description and Usage

The Upgrade tab provide users focus areas and budget constraints to receive tailored hardware upgrade paths that maximize HFT performance within financial limits. This tab rendered by render_upgrade_tab in upgrade_tab.py collects focus areas as multi select and optional budget.

Redfish inclusion is supported. `perform_upgrade_analysis` aggregates focused sections computes dynamic snapshot and builds preview text.

On run it passes budget level to the prompt instructing the AI to scale recommendations from basic to premium 2026 parts while enforcing hardware compatibility from the summary. The structured task stores `last_upgrade`. Display shows analysis cost metrics for selected upgrades total and generates `hft_upgrade_report.md` via `generate_selected_report` which includes only checked items.

3.6. AI Settings Tab Description and Usage

The AI Settings tab allows users to configure their preferred AI backend here to power all analysis features across the app. The tab from `render_ai_settings_tab` in `ai_settings_tab.py` displays current config values. Text inputs and sliders allow changes to model, api key, base url, temperature, `top_p` and max tokens. The save button writes to `ai_config.json` via `save_ai_config` and updates session state. The test button calls `test_ai_connection` which performs a minimal `litellm` completion and reports success or error.

4. Supporting Functions and Technical Details

Supporting functions across files include `count_tokens` in `data.py` which approximates token count by dividing text length by four for prompt size feedback. `build_redfish_context` formats selected Redfish sections for AI injection. All AI interactions use the universal `render_ai_chat` and `render_structured_ai_task` for consistency and streaming UX. The `render_ai_chat` function manages per tab message lists in session state handles streaming responses from `litellm` and error display. AI calls are abstracted through `litellm` allowing seamless switching between local Ollama instances and cloud providers like Grok, OpenAI, or Anthropic by changing the model string and credentials in settings. Structured JSON enforcement with fallback parsing ensures reliable extraction of recommendations that translate directly into executable tuning scripts or checklists. Context previews and manual editing empower users to refine AI inputs for more accurate domain specific advice.

The application maintains state through Streamlit session state for sections redfish data analysis results chat histories and monitoring state. This enables seamless tab switching and rerun resilience.

5. Conclusion

In summary this integrated platform streamlines the full lifecycle of HFT system optimization delivering data driven AI insights that translate into concrete low latency improvements across operating system, firmware, compiler and application layers plus forward looking hardware upgrade guidance. Users are encouraged to begin with the Data tab collect information then monitor key metrics and proceed through Performance and Upgrade tabs for comprehensive optimization.